

Matthew Phillips

Software Integration Engineer | Vehicle Software | Firmware + ECU Integration | CAN / Embedded Systems

bajaracer415@gmail.com | 415-504-5897

SUMMARY

Senior vehicle-software and product engineer with hands-on experience building hardware-connected automotive systems across firmware-adjacent workflows, CAN bus telemetry, BLE device communication, diagnostics, datalogging, software release behavior, mobile tooling, and full-stack support/operations systems. I am strongest in integration-heavy environments where the problem crosses boundaries: firmware, device communication, vehicle signals, logs, user-facing tools, backend systems, and the people trying to debug what happened.

For Harbinger, the fit is the integration layer: I have built and debugged software around real vehicle hardware including JB4 tuning devices, CANFlex fuel/sensor controllers, and a TurboLamik AWD/transmission telemetry project. That work includes CAN signal mapping, protocol reverse engineering, firmware update workflows, raw frame capture, telemetry decoding, device health/reconnect behavior, diagnostic support tooling, documentation, and safety-first validation before control output.

CORE STRENGTHS

- Vehicle software integration: CAN bus signal decoding, CAN FD/CAN-oriented vehicle network workflows, ECU/device interfaces, BLE, serial-style protocols, telemetry parsing, diagnostics, datalogging, and configuration tooling.
- Firmware and embedded-adjacent workflows: ESP-IDF-oriented firmware scaffolding, CAN RX integration points, watchdog/health state, raw frame capture, bootloader/firmware update UX, HEX parsing, checksum validation, and release/version behavior.
- System-level debugging: log analysis, vehicle data intake, raw/decode comparison, protocol mapping, connection recovery, trace-style diagnostics, customer/field issue reproduction, support escalation, and root-cause documentation.
- Automotive domain depth: OBD2-connected hardware, ECU/tuning workflows, fuel/ethanol sensing, injector/fuel-system control, transmission/TCU telemetry, wheel speed, gear, torque, slip, boost, RPM, AFR, meth/WMI, E85, and vehicle platform/firmware variation.
- Integration automation and tooling: Python scripting, Node.js/TypeScript tooling, data pipelines, admin dashboards, support portals, APIs, WebSockets, logging, analytics, AI-assisted triage, and repeatable diagnostic workflows.
- Practical engineering habits: scoped changes, readable code, careful documentation, testable paths, cross-functional communication, clear handoffs, release awareness, and ownership of production behavior.

PROFESSIONAL EXPERIENCE

Burger Motorsports

Automotive Tuning, Vehicle Software, Firmware-Adjacent Integration, CAN Bus, BLE, Mobile Development

2019 onward

- Helped build and launch the JB4 tuning device ecosystem, including device-facing software, firmware-related workflows, mobile application features, customer tools, diagnostics, support systems, and platform-specific behavior across many vehicle platforms and firmware combinations.
- Built JB4Pro mobile/device software using Flutter/Dart with BLE scan/connect behavior, connection recovery, notification parsing, ASCII/byte-level command handling, live gauges, datalogging, settings screens, user-adjustment interfaces, and developer diagnostics.
- Implemented firmware update workflows for JB4Pro-style hardware, including firmware download/manual HEX loading, bootloader entry, erase/write commands, block flashing, checksum validation, exit bootloader handling, progress state, and user-facing safety warnings.
- Built and maintained command/protocol logic for JB4 hardware, including settings retrieval, settings save, logging start/pause, map selection, E85/flex fuel configuration, WMI/meth settings, boost/fuel tables, AFR/meth/fuel pressure telemetry, and developer packet inspection.
- Developed CANFlex-style hardware software for fuel and sensor control: BLE device communication, ethanol content, fuel temperature, fuel pressure, CANbus output configuration, analog output modes, pressure sensor modes, calibration controls, firmware/version display, live gauges, logs, and race-car tuning workflows.
- Worked close to embedded systems, firmware, custom hardware, and vehicle communication protocols, translating raw device/vehicle data into reliable UI state and supportable diagnostic workflows.
- Built CRM-backed support tooling that replaced manual email-based log review with structured customer submissions, vehicle log intake, AI-assisted parsing, diagnostic/tuning guidance, support history, and escalation paths.
- Operated in a domain where incorrect parsing or firmware/version assumptions can affect real vehicles, requiring careful validation, documentation, and ownership of edge cases.

Independent Vehicle/Hardware Software Projects

TurboLamik AWD Controller, CANFlex, JB4Pro, Vehicle Telemetry and Integration Tooling

- Built a TurboLamik AWD controller project with ESP-IDF-oriented firmware scaffolding for BMW E90 + TurboLamik integration, including passive CAN listening on a shared 500 kbps bus, selected E90 CAN ID decoding, TurboLamik TCU frame decoding, normalized drivetrain/chassis signals, derived metrics, health/watchdog state, raw CAN capture, and shadow-mode AWD request computation.

- Reverse engineered TurboLamik ADX protocol behavior into a client-device spec, including 921600 baud serial assumptions, LOG1/LOG2 polling sequence, fixed 100-byte packet maps, gearbox torque, engine torque, input/output RPM, slip/clutch data, linear pressure, lockup, analog inputs, CAN torque fields, AWD activation, PWM output, steering angle, and error/safe-mode data.
- Built vehicle signal maps for BMW E90 PT-CAN-style traffic, including throttle, engine RPM, wheel speeds, coolant/oil temperature, steering angle, brake proxy, handbrake, reverse, and validation notes separating confirmed decodes from provisional in-car validation items.
- Documented MaxxECU default CAN output integration for 11-bit IDs 0x520-0x542 at 500 kbit/s, including RPM, throttle, MAP, lambda, ignition, fuel duty, vehicle speed, slip, traction flags, brake/clutch flags, accelerations, oil pressure/temp, boost target, transmission temp, differential temp, and firmware-version-gated protocol differences.
- Designed the Flutter telemetry/logging app shell for the vehicle side, including device status, live dashboard, raw frame monitor, decoded signal monitor, capture control, saved sessions/export, and vehicle profile display.
- Built diagnostic and support workflows around hardware logs, customer-submitted traces, firmware/platform context, parameter parsing, and escalation paths for human review.

Freelance Full-Stack Product Engineer / Integration Developer

Automation, Internal Tools, AI Workflows, Customer Portals, Operational Systems

Current

- Build full-stack applications and integrations for Stealth Batteries and related businesses, owning ambiguous product problems from discovery through architecture, implementation, debugging, deployment, and iteration.
- Built Stealth Batteries commerce/admin systems using Next.js, React, TypeScript, Payload CMS, Postgres, Stripe, webhooks, Resend/Nodemailer, Vercel Blob, UPS/shipping APIs, dealer portals, affiliate workflows, order management, warranty/support workflows, and admin dashboards.
- Built AI-assisted support and autonomous workflow systems for customer support, onboarding, warranty/claims handling, internal workflow assistance, knowledge-base context, logging, analytics, and escalation paths.
- Built CRM/order/customer tracking systems with Node/Express, SQLite, JWT auth, role permissions, admin approval workflows, customer portals, OCR/document scanning, real-time updates, Twilio/Nodemailer notifications, audit trails, and deployment documentation.
- Use AI development tools daily for implementation, review, documentation, and debugging, while retaining responsibility for correctness, security, and production behavior.

Bayer

Technical / Analytical Testing Software Background

2015-2018

- Worked in analytical/scientific environments after studying biochemistry and software engineering.
- Built and supported custom code and analytical workflows for regulated testing and scientific/technical operations.
- Gained experience with careful documentation, data integrity, regulated processes, quality expectations, and technical troubleshooting.

SELECTED PROJECT HIGHLIGHTS

TurboLamik AWD / Transmission Telemetry and Integration

Built an AWD controller and telemetry foundation around BMW E90 + TurboLamik, including CAN RX scaffolding, E90 decoder, TurboLamik decoder, raw CAN capture, health/watchdog state, derived metrics, shadow-mode AWD request logic, BLE service stub, Flutter live dashboard, raw frame monitor, decoded signal monitor, and saved log/export workflows.

JB4Pro Vehicle Tuning Platform

Built mobile and hardware-connected software for JB4Pro tuning hardware, including BLE device discovery/connection, command handling, live telemetry, logging, firmware update flows, settings save/retrieval, map/user adjustment screens, E85/flex fuel workflows, WMI/meth configuration, boost/fuel tables, AFR/fuel pressure telemetry, and developer diagnostics.

CANFlex Fuel and Sensor Controller

Built CANFlex software for a fuel injector / E85 sensor / fuel pressure control ecosystem with BLE communication, ethanol/fuel-temperature/fuel-pressure gauges, CANbus and analog output configuration, pressure sensor modes, calibration, logs, firmware/version handling, and ECU/JB4 integration paths for race-car tuning.

TECHNOLOGIES

Vehicle / Embedded / Protocols: CAN bus, CAN-oriented ECU/device workflows, OBD2-connected hardware, BLE, serial-style protocols, ESP-IDF scaffolding, firmware update flows, bootloader workflows, HEX parsing, checksums, telemetry decoding, diagnostics, datalogging, live gauges, raw frame capture, watchdog/health state, trace/log analysis.

Integration / Tooling: Python, scripting, Git, documentation, release/version awareness, diagnostic workflows, replay-style log review, customer/field issue triage, AI-assisted support tooling, APIs, WebSockets, dashboards, automation.

Languages / Frameworks: C, C++, C#, Dart/Flutter, Java/Android concepts, Python, TypeScript, JavaScript, Node.js, Express, React, Next.js.

Backend / Data / Ops: Postgres, SQLite, MongoDB/Mongoose, Payload CMS, Supabase, Redis/BullMQ-style queues, REST APIs, JWT, role permissions, Stripe, shipping APIs, email/SMS providers, logging, analytics, admin dashboards.

EDUCATION

Biochemistry and Software Engineering coursework, 2012-2015 Studied biochemistry with a software engineering focus while preparing to build analytical testing and technical workflow software.

HOW I WORK

I am comfortable with “here is the integration problem, go figure it out” work. I ask scope questions early, especially around interfaces, diagnostics, edge cases, ownership, safety, and production risk. I prefer simple architecture, clear data flow, validation before control output, and documentation that lets the next engineer reproduce the issue instead of guessing.